

M1.1: Rustls 内存安全设计架构

Theory: 基于 Rust 语言的强类型系统与所有权机制，在编译期杜绝了缓冲区溢出、Use-After-Free 等 C 语言常见的内存安全漏洞。

Details: 完全使用 Safe Rust 编写核心数据结构，禁止直接操作未初始化内存与裸指针。通过精简协议支持（仅限 TLS 1.2/1.3）将代码量降低至 OpenSSL 的几分之一。

Trade-off: 不支持过时的加密套件和历史版本，无法向后兼容使用古老加密算法（如 3DES、RC4）的遗留设备，可能导致旧硬件建连失败。

M1.2: TLS 1.3 握手协议与状态机

Theory: 采用代数数据类型强约束握手机制，实现单次往返（1-RTT）即可完成安全通道建立。

Details: 在 'rustls::conn::HandshakeState' 中定义状态机的强类型枚举，利用所有权消费旧状态并产生新状态，从编译层面杜绝握手重放与步骤跨越攻击。

Trade-off: 严格的状态迁移使得握手期间若收到非预期的消息，会触发 UnexpectedMessage 致命警报，无法像 OpenSSL 那样通过宽松容错继续处理。

M1.3: ClientHello 与 ServerHello 结构

Theory: 交换客户端与服务端的基础支持项，通过密码套件和 KeyShare 实现安全共识。

Details: 客户端通过 ClientHello 发起包含支持的 CipherSuites、支持的组（Supported Groups）及密钥共享（KeyShare）。服务端回复 ServerHello 确认选定项。

Trade-off: 包含 SNI（服务器名称指示）明文，中间人可能窥探访问域名。建议未来集成 ECH（加密客户端 Hello）保护隐私。

M1.4: 密码学适配接口与 CryptoProvider

Theory: 抽象密码底层计算逻辑，支持可插拔的硬件或软件密码加速器。

Details: 引入 'rustls::crypto::CryptoProvider' 特性，定义非对称签名、对称 AEAD 及密钥交换接口。默认绑定 *ring* 或 AWS-LC 作为后端。

Trade-off: 灵活的可插插接口增加了模块化开销，且如果第三方密码引擎实现有漏洞，将破坏 Rustls 的整体安全性。

M1.5: 密钥派生与 HKDF 机制

Theory: 基于 HMAC 的解构-扩展密钥派生算法，从协商的共享秘密中生成多种握手与应用密钥。

Details: 根据 RFC 5869, 分 HKDF-Extract 与 HKDF-Expand 两步。使用 Transcript Hash 和标签（Label）生成读写加密密钥及 Finished 验证码。

Trade-off: 密钥派生计算密集，需要高强度哈希算法（如 SHA256/384）支持，在高并发短连接场景下对 CPU 算力要求较高。

M1.6: 会话恢复与预共享密钥 (PSK)

Theory: 复用既有安全上下文，缩短后续建连时对称密钥协商的计算开销。

Details: 握手成功后生成 PSK 并关联唯一的 Resumption ID。新握手时客户端在 ClientHello 中注入 PSK 扩展项，服务端匹配后可跳过证书校验。

Trade-off: 若 PSK 长期不变且未引入 ECDHE 混淆，则一旦泄露将失去前向安全性（Forward Secrecy），应结合 DHE 组合使用。

M2.1: 会话票据 (Session Tickets) 分发

Theory: 服务端向客户端颁发包含加密会话状态的票据，实现无状态的分布式恢复。

Details: 服务端在握手后发送 NewSessionTicket，由服务端主密钥加密。客户端保存此票据并在下次 ClientHello 的 psk_identity 中呈现。

Trade-off: 服务端主密钥需要定期轮换，如果主密钥被窃取，攻击者可以解密所有关联的历史会话票据。

M2.2: 记录协议与记录加密格式

Theory: 结构化传输信道，包装所有高层协议数据并为其实施切片与对称加密。

Details: 在 'rustls::record_layer::Record' 中定义包头。头部包含 1 字节类型、2 字节 TLS 版本和 2 字节长度，后跟加密后的有效载荷和认证签名。

Trade-off: 包头长度及填充方式可能暴露真实载荷的大致大小，需配置零填充（Padding）来模糊流量特征。

M2.3: AEAD 对称加密

Theory: 关联数据认证加密机制，同时保证数据的机密性与完整性，防止篡改。

Details: 弃用传统的“先加密后 MAC”或“先 MAC 后加密”，强制使用 AES-128/256-GCM 或 ChaCha20-Poly1305，利用单条指令完成数据加密与认证。

Trade-off: 必须严格保证 Nonce（临时随机数）的唯一性，重复的 Nonce 会导致密钥流泄漏，从而完全破坏 AEAD 加密安全性。

M2.4: Transcript Hash 握手哈希校验

Theory: 捕获并单向哈希化全部握手数据流，防止握手协商参数被中间人降级或篡改。

Details: 握手期间将发送和接收的所有原始字节输入 'rustls::hash__handshake::HandshakeHash'，在握手结束时用于派生 Finished 校验和。

Trade-off: 需要在内存中实时维护一段 Transcript 缓冲区，虽然开销极小，但在大并发建连时会增加一些内存垃圾回收压力。

M2.5: 证书链验证与 webpki 集成

Theory: 基于 WebPKI 标准验证服务端身份，验证签名合法性及信任链拓扑。

Details: 将验证逻辑剥离至独立的 'rustls-webpki' 库，严密解析 X.509 证书的 DER 编码，执行主机名匹配、有效期校验及签名路径验证。

Trade-off: 默认不使用操作系统自带的 OpenSSL 证书存储，需要额外配置 'rustls-native-certs' 或打包 'webpki-roots'，增加了部署复杂度。

M2.6: 客户端证书验证与双向 TLS

Theory: 允许服务端要求并验证客户端证书，实现点对点的强双向身份校验。

Details: 服务端发送 CertificateRequest。客户端回复自己的 Certificate 与 CertificateVerify，其中包含对当前握手哈希的签名。

Trade-off: 双向认证会显著增加握手耗时与包体积，且需要完备的客户端证书分发与撤销（CRL/OCSP）管理机制。

M3.1: 签名方案协商

Theory: 协商双方共同支持的非对称算法与哈希组合，用于握手数字签名与验证。

Details: 协商 'SignatureScheme'（如 'ECDSA_NISTP256_SHA256'、'RSA_PSS_RSAE_SHA256'）。客户端明示支持列表，服务端选用并以此方案对 KeyShare 进行数字签名。

Trade-off: 废弃了安全性低下的 MD5-RSA、SHA1-RSA 等组合，导致不支持强签名算法的历史设备完全无法连接。

M3.2: 密钥交换算法执行

Theory: 利用椭圆曲线或有限域 Diffie-Hellman 算法，在不安全信道上安全协商出对称密钥。

Details: 采用临时密钥交换 (Ephemeral)。客户端与服务端独立生成私钥并计算公钥，通过 'SupportedGroups' 传递，最终计算出相同的 Pre-master Secret。

Trade-off: 密钥交换运算需做高开销的模幂或椭圆曲线乘法，是 TLS 握手最耗时的计算步骤，易遭受 CPU 耗尽型 DoS 攻击。

M3.3: 零往返延迟 (0-RTT) 早期数据

Theory: 客户端在第一次发送握手数据包时就携带加密的应用数据，实现零延迟发送。

Details: 客户端利用之前会话建立的 PSK 密钥加密早期数据 (Early Data)，将其放入 ClientHello 后的数据流中。

Trade-off: 0-RTT 数据无法防止重放攻击 (Replay Attack)，必须结合服务端单向窗口校验或防重放过滤器，且只能用于幂等的 GET 请求。

M3.4: 加密扩展传输

Theory: 将与密钥协商无关的服务端扩展信息移入加密通道，降低明文泄漏暴露风险。

Details: 在 ServerHello 完成临时共享秘钥派生后，服务端发送 EncryptedExtensions 消息。该消息使用握手加密密钥加密传输，包含 ALPN 等扩展。

Trade-off: 增加了握手过程的解析层级，由于 ALPN 加密，中间设备无法通过旁路抓包进行应用协议统计。

M3.5: 动态密钥更新 (KeyUpdate)

Theory: 在同一个长期 TCP 连接中动态更换对称密钥，防止因单个密钥加密过多流量而被破解。

Details: 发送 'KeyUpdate' 握手消息，重新进行 HKDF 派生，更新当前方向上的 Application Traffic Secret。接收方收到后同步更新解密密钥。

Trade-off: 频繁的 KeyUpdate 会带来轻微的延迟，且实现不当可能导致通信双方的加解密步调不一致，引发连接中断。

M3.6: 警报协议错误处理

Theory: 提供标准化异常信号机制，让对端明确感知关闭原因或协议冲突。

Details: 警报由 1 字节级别 (Warning/Fatal) 和 1 字节描述码 (如 HandshakeFailure) 组成。Fatal 级别警报会立即彻底关闭 TLS 状态机。

Trade-off: 过于详细的警报可能向潜在攻击者透露系统内部状态 (如证书无效的具体步骤)，需要折中控制错误暴露细节。

M4.1: 规避 Heartbleed 心跳设计

Theory: 严格裁剪不必要的协议特性，在代码编写层面从根本上消除危险的数据回显逻辑。

Details: Rustls 彻底不实现 TLS Heartbeat 扩展 (RFC 6520)。此外，Rust 的安全切片与边界检查保证了即使发生内存读取请求，也不会越界读取邻近内存。

Trade-off: 彻底放弃 Heartbeat 意味着无法使用原生的应用层心跳探活，必须在 TCP 层面 (Keep-Alive) 或应用层协议中自行实现探活。

M4.2: 中间盒兼容模式

Theory: 伪装握手机制，防止不支持 TLS 1.3 的中间路由因防火墙规则直接丢包。

Details: 在 ClientHello 中注入随机的 32 字节 Legacy Session ID，并在 ServerHello 后发送一条伪造的明文 ChangeCipherSpec (CCS) 消息。

Trade-off: 兼容模式会在握手流中多出无意义的 CCS 字节包，虽解决了穿透问题，但浪费了些许的网络带宽和解析性能。

M4.3: Finished 结束校验消息计算

Theory: 验证双方计算的对称密钥是否一致，且整段握手消息流在传输中未被篡改。

Details: Finished 消息由 HMAC 计算，其 Key 由 HKDF 派生，输入数据为限制在握手范围内的 Transcript Hash。若接收端算出的 HMAC 不匹配则握手失败。

Trade-off: Finished 校验是握手成败的最终关卡，若前置的证书签名或密钥派生出错，均会在本步骤抛出解密失败，排查故障较困难。

M4.4: 侧道时间攻击防范 (Constant-Time)

Theory: 消除敏感密码学计算的执行耗时与输入数据之间的依赖关系，防止攻击者通过时序推算密钥。

Details: 所有的 MAC 比对、签名验证均调用常数时间比较库 (如 subtle 库)。使用无短路逻辑的位运算替代常规的 'if' 循环比较。

Trade-off: 常数时间比对会牺牲“快速失败”的优化，即便首个字节不匹配也必须扫描完全部数据，导致非法请求的响应延迟微幅上升。

M4.5: Rustls Config API 设计模型

Theory: 贯彻 Rust 的“不可变性”与“类型安全”原则，提供防误用的客户端与服务端配置 API。

Details: 'ClientConfig' 和 'ServerConfig' 使用 Builder 模式。底层使用 'Arc' 实现多线程安全共享。一旦调用 'build()' 即被冻结，防止运行时被修改。

Trade-off: 配置对象一旦构建完毕就无法动态更改 (如热更新证书)，需要通过更换 Arc 指向或动态解析证书解析器来实现热重载。

M5.1: 异步 I/O (tokio-rustls) 异步集成

Theory: 将同步握手和读写状态封装为非阻塞的 Future 驱动模型，适合高并发网络服务。

Details: 'tokio-rustls' 将 Rustls 的同步 Connection 包装成 'TlsStream'，实现 'tokio::io::AsyncRead' 与 'AsyncWrite'，在 epoll 就绪时推进握手。

Trade-off: 异步调度引入了上下文切换和 Pin 包装，在极端单核吞吐量测试中，其性能可能略低于高度调优的裸同步多线程模型。

M5.2: 证书锁 (Certificate Pinning) 与信任锚点

Theory: 锁定预期的证书指纹或特定 CA，绕过系统的全局 CA 信任链，防范恶意 CA 签发伪造证书。

Details: 实现自定义的 'ServerCertVerifier' 特性，硬编码目标证书公钥的 SHA256 哈希，在握手期强行截获并只通过匹配指纹的证书。

Trade-off: 证书一旦过期或服务器更换了 CA，客户端若未及时热升级新硬编码的指纹，将直接导致业务彻底中断。

M5.3: 模糊测试与安全审计机制

Theory: 通过大流量变异输入与差异化测试，验证代码解析极端畸形包时的鲁棒性，保障零崩溃。

Details: 集成 'cargo-fuzz' (基于 libFuzzer)，对 TLS 记录解码器、ClientHello 解析器进行百亿次模糊输入测试，确保无 Panic 与 OOM。

Trade-off: 编写与运行模糊测试需要大量的 CPU 算力与测试用例积累，且对异步宏等复杂业务流的测试覆盖面有限。

M5.4: 基准测试与性能调优对比

Theory: 使用精细的微基准测试评估吞吐率与建连时延，识别内存分配与密码计算瓶颈。

Details: 引入 Criterion 基准测试，优化握手期间的动态内存分配 (使用 'Vec::with_capacity')，并采用零拷贝切片设计直接从 TCP 接收缓冲读取数据。

Trade-off: 过度追求性能可能导致代码可读性下降，例如大量使用生命周期引用替代 Arc，增加后续重构的认知难度。

M5.5: 后量子密码学密钥协商

Theory: 引入抗量子计算攻击的非对称算法，在未来量子计算机成熟时提供坚实的密钥协商防线。

Details: 支持 x25519+ML-KEM-768 (原 Kyber) 等混合密钥交换方案。客户端在 KeyShare 中同时发送经典与 PQC 公钥，执行双重密钥协商。

Trade-off: PQC 公钥与密文尺寸极大 (通常为经典算法的数倍)，会导致 ClientHello 发生 IP 分片，增加网络丢包和握手延迟。

Zone T1: Rustls Core APIs

rustls::conn::Connection, rustls::conn::HandshakeState,

rustls::crypto::CryptoProvider,

rustls::record_layer::RecordLayer,

rustls::hash_handshake::HandshakeHash,

rustls::key_schedule::KeySchedule

Zone T2: Protocol & Verification Errors

tls_handshake_unexpected_message,

tls_record_decrypt_failed, webpki_cert_path_invalid,

tls_handshake_transcript_mismatch, constant_time_comparison_leak, zero_rtt_replay_detected